

Disciplina: Análise e complexidade de algoritmos

Nome da Atividade: Grupo 4 — Limite Inferior para Ordenação e Ordenação em Tempo Linear

Participantes da equipe:

Arthur Christopher Pires Dutra – Bi3025616;

Diego Pires Goncalves – Bi3025845;

Pedro Henrique Tamburi Sinhorini – Bi3024555;

Raissa Santana da Silva – Bi3024954

Github: <https://github.com/RaissaSantana13/algorithm-analysis-project>

Introdução:

A ordenação de dados é um pilar da Engenharia da Computação. Ela aparece em quase tudo o que usamos, desde a organização de bancos de dados até sistemas que permitem buscas rápidas e algoritmos de geometria. Como já existem métodos muito eficientes, como o *Merge Sort* e o *Heapsort*, que resolvem esse problema com complexidade $O(n \log n)$, é natural nos perguntarmos se existe algum algoritmo ainda desconhecido que consiga ser ainda mais rápido, talvez chegando ao tempo linear.

Este relatório analisa essa questão focando nos algoritmos baseados em comparações, onde a única forma de entender a ordem dos dados é comparando um elemento com outro. O objetivo aqui é mostrar que esses algoritmos possuem um limite que não pode ser superado, que é o de $\Omega(n \log n)$ comparações no pior cenário possível. Diferente de uma cota superior, que avalia a eficiência de um algoritmo específico, estamos tratando de uma característica do próprio problema. Para provar isso, usaremos o modelo de árvore de decisão, que simplifica a análise e transforma o problema de complexidade em um cálculo de altura de uma árvore binária.

Cota Superior e Cota Inferior:

É essencial distinguir os dois tipos de limite assintótico que aparecem no estudo de ordenação, pois eles se referem a entidades distintas. Uma **cota superior** $O(f(n))$ é uma propriedade de um *algoritmo concreto*: afirma que aquele procedimento específico nunca executa, no pior caso, mais que $c \cdot f(n)$ operações, para alguma constante c e n suficientemente grande. Quando dizemos que o *Merge Sort* é $O(n \log n)$, fazemos uma garantia sobre o comportamento daquele algoritmo, nada impede que outro algoritmo seja mais rápido.

Já uma **cota inferior** $\Omega(g(n))$ é uma propriedade do *problema*: afirma que todo algoritmo, dentro do modelo considerado, é obrigado a executar pelo menos $c' \cdot g(n)$ operações em alguma entrada de tamanho n . O quantificador é universal, a cota inferior é, em essência, um teorema de impossibilidade, que descarta de uma só vez toda a infinidade de algoritmos imagináveis, inclusive os ainda não inventados. Por isso, demonstrar uma cota inferior é qualitativamente mais difícil do que exibir uma cota superior: não basta apresentar um exemplo eficiente; é preciso argumentar sobre a totalidade do espaço de soluções.

Quando a **cota superior** do melhor algoritmo conhecido encontra a **cota inferior** do problema, o algoritmo é dito **assintoticamente ótimo**. Para a ordenação por comparação, mostraremos que ambos os limites valem em $n \log n$, de modo que o problema é $\Theta(n \log n)$ e algoritmos como *Merge Sort* e *Heapsort* são, neste modelo, o melhor que se pode obter a menos de fatores constantes.

Demonstração Formal do Limite Inferior

Para estabelecer um resultado válido para qualquer algoritmo de comparação, abstraímos o algoritmo por meio de uma **árvore de decisão**. Fixado n , trata-se de uma árvore binária em que cada nó interno representa uma comparação entre dois elementos da entrada, a_i, a_j , e as duas arestas descendentes correspondem aos dois desfechos possíveis dessa comparação. Cada folha é rotulada por uma permutação dos índices e representa a resposta emitida pelo algoritmo. A execução sobre uma entrada específica equivale a um caminho da raiz até uma folha, e o número de comparações no **pior caso** coincide exatamente com a **altura** h da árvore, o caminho raiz–folha mais longo. Assim, limitar inferiormente o pior caso reduz-se a limitar inferiormente h .

O passo inicial é contar quantas folhas precisamos. Sabemos que existem $n!$ formas diferentes de ordenar n elementos. Como cada uma dessas formas exige uma resposta única, duas entradas diferentes não podem terminar na mesma folha. Se isso acontecesse, a folha entregaria a mesma resposta para ordens distintas, o que está errado. Portanto, a árvore precisa ter pelo menos ℓ folhas. Sabemos também, por uma propriedade básica das árvores binárias, que uma árvore com altura h tem no máximo 2^h folhas. Juntando essas duas informações, temos que $\ell \leq 2^h$ precisa ser pelo menos $n! \leq \ell \leq 2^h$. Tirando o logaritmo na base 2 dos dois lados, chegamos a $h \geq \log_2(n!)$.

Agora, precisamos entender como esse valor cresce. Usando a aproximação de Stirling, que descreve o fatorial para números grandes, o logaritmo de $\log_2(n!)$ é aproximadamente $\ln(n!) = n \ln n - n + O(\ln n)$. Quando convertemos isso para a base 2, vemos que o termo que realmente importa é o $\log_2 x = \ln x / \ln 2$. Isso nos mostra que o logaritmo de $\log_2(n!) = \Theta(n \log n)$ é da ordem de $\log_2(n!) = \Omega(n \log n)$. Como nossa altura $h = \Omega(n \log n)$ é pelo menos esse valor, concluímos que $\Omega(n \log n)$ cresce pelo menos na proporção de $O(n \log n)$. Ou seja, nenhum algoritmo de comparação consegue ordenar elementos fazendo menos de $\Theta(n \log n)$ comparações no pior caso. Como o Merge Sort e o Heapsort já atingem esse limite, podemos afirmar que eles são algoritmos ótimos dentro desse modelo.

Counting Sort:

O Counting Sort é um algoritmo de ordenação não baseado em comparações que explora a estrutura e a natureza dos dados de entrada. Diferente de algoritmos convencionais, ele não questiona se um elemento é maior ou menor que outro; em vez disso, constrói um vetor de frequências de cada valor distinto presente. A partir dessas contagens acumuladas, ele deduz a posição exata de cada elemento no vetor final ordenado. O processo lógico ocorre em etapas sequenciais: primeiro, determina-se o valor máximo na entrada (denominado k), o que define o tamanho do vetor auxiliar de contagem. Em seguida, o algoritmo registra a frequência de cada elemento. Esse vetor é então transformado por meio de uma soma de prefixos, onde cada posição passa a indicar a quantidade exata de elementos que são menores ou iguais àquele índice. Finalmente, o vetor original é percorrido para alocar cada dado diretamente em seu índice definitivo.

Para implementar essa lógica, o código realiza mapeamentos diretos na memória. Abaixo, a representação estrutural utilizada para os testes experimentais da equipe:

```
1 public static void CountingSort(int[] arr)
2 {
3     int n = arr.Length;
4     if (n <= 1) return;
5
6     int max = arr[0];
7     for (int i = 1; i < n; i++)
8         if (arr[i] > max) max = arr[i];
9
10    int[] count = new int[max + 1];
11    int[] output = new int[n];
12
13    for (int i = 0; i < n; i++) count[arr[i]]++;
14    for (int i = 1; i <= max; i++) count[i] += count[i - 1];
15
16    for (int i = n - 1; i >= 0; i--)
17    {
18        output[count[arr[i]] - 1] = arr[i];
19        count[arr[i]]--;
20    }
21
22    for (int i = 0; i < n; i++) arr[i] = output[i];
23 }
```

Fonte: Autoria própria

A análise temporal revela a força matemática deste método para escapar da cota inferior de $\Omega(n \log n)$. A inicialização do vetor de contagem exige tempo $\Theta(k)$. A varredura dos elementos e a alocação final demandam tempo $\Theta(n)$. Consequentemente, o tempo de execução do algoritmo é rigorosamente $\Theta(n + k)$. Entretanto, esse desempenho linear possui uma restrição severa atrelada à memória espacial, que também é de $O(n + k)$. Se o intervalo dos dados (k) for absurdamente maior que a quantidade de elementos (n), o algoritmo exigirá a alocação de um vetor gigante apenas para o mapeamento. Nesse

cenário, o consumo de memória dispara exponencialmente e a eficiência desmorona, pois o tempo gasto apenas varrendo posições vazias de k ofuscaria a ordenação em si.

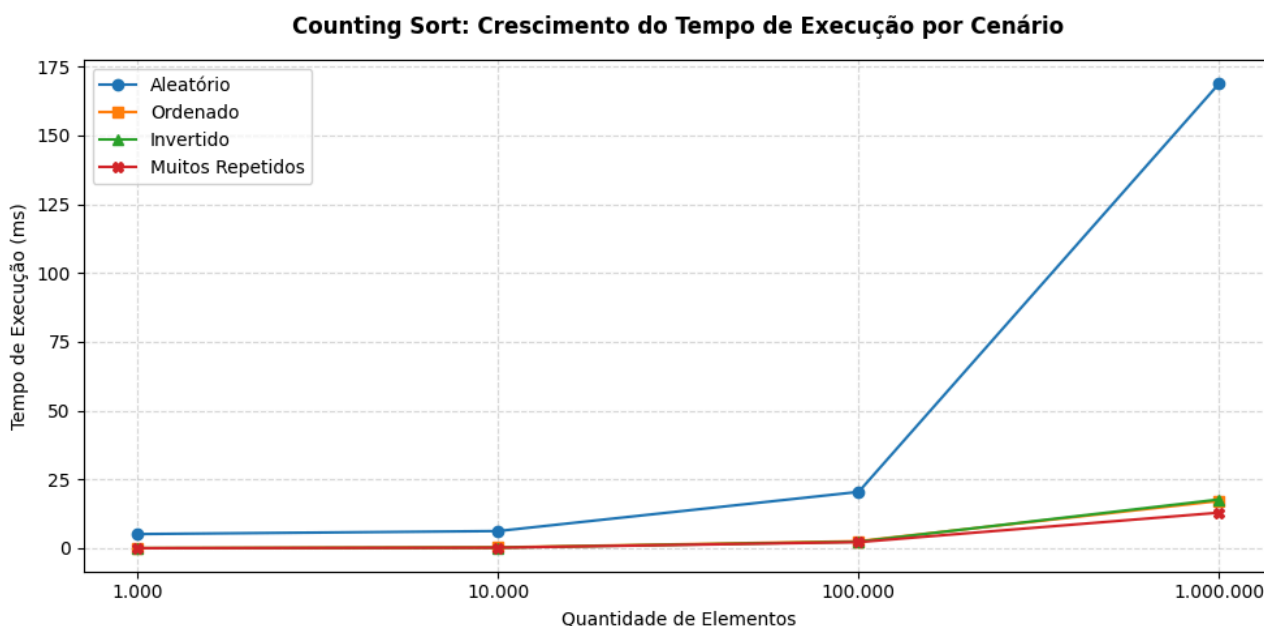
Outra característica arquitetural vital do Counting Sort é a sua estabilidade — a garantia de que elementos com chaves repetidas mantêm estritamente a mesma ordem relativa com que apareceram na entrada. Para garantir essa propriedade matemática, a etapa final de distribuição (o último laço **for** do código) deve obrigatoriamente iterar de trás para frente. É por conta dessa estabilidade absoluta que o Counting Sort é a escolha padrão para atuar como a sub-rotina interna do Radix Sort ao organizar dados dígito a dígito.

Para traduzir a teoria em dados mensuráveis, o algoritmo foi submetido a baterias de testes práticos utilizando diferentes distribuições de entrada: vetores com valores aleatórios, vetores ordenados, vetores em ordem invertida e vetores com altíssima densidade de elementos repetidos. O limite máximo de valor (k) foi mantido em uma proporção viável para a alocação de memória do sistema. Os tempos médios de execução foram organizados na tabela abaixo:

Quantidade de Elementos	Aleatório	Ordenado	Invertido	Muitos Repetidos
1.000	5,15 ms	0,03 ms	0,02 ms	0,02 ms
10.000	6,24 ms	0,30 ms	0,21 ms	0,20 ms
100.000	20,48 ms	2,56 ms	2,36 ms	2,22 ms
1.000.000	168,91 ms	17,19 ms	17,73 ms	12,93 ms

Fonte: Autoria própria

A tabela evidencia a natureza não-comparativa do modelo. Diferente de algoritmos de particionamento, o Counting Sort é "cego" para a disposição inicial dos dados. Os tempos de processamento mantiveram-se quase idênticos para as entradas aleatórias, ordenadas ou invertidas, pois o custo computacional está atrelado apenas a operações diretas de leitura e escrita em memória. O comportamento visual dessa escala pode ser observado no gráfico a seguir:



Fonte: Autoria própria

O destaque absoluto de desempenho ocorreu no cenário composto por "Muitos Repetidos". Nesse tipo de distribuição, o intervalo de variação k é ínfimo. Como resultado, o custo temporal da alocação de memória e da soma de prefixos cai drasticamente, consolidando o processamento em um comportamento linear $\Theta(n + k)$ altamente eficiente e superando largamente a barreira teórica dos algoritmos comparativos.

Radix Sort:

O Radix Sort é um algoritmo de ordenação não comparativo que organiza os elementos processando seus dígitos individualmente. Diferentemente de algoritmos como Quick Sort ou Merge Sort, ele não compara diretamente os valores entre si. Em vez disso, os números são agrupados e reorganizados de acordo com cada dígito, normalmente começando pelo menos significativo (LSD - Least Significant Digit) e avançando até o mais significativo.

Para realizar essa ordenação, o Radix Sort geralmente utiliza o Counting Sort como algoritmo auxiliar em cada etapa. Inicialmente, os elementos são organizados de acordo com o dígito das unidades. Em seguida, o processo é repetido para as dezenas, centenas e assim por diante, até que todos os dígitos tenham sido processados. Ao final, os elementos encontram-se completamente ordenados.

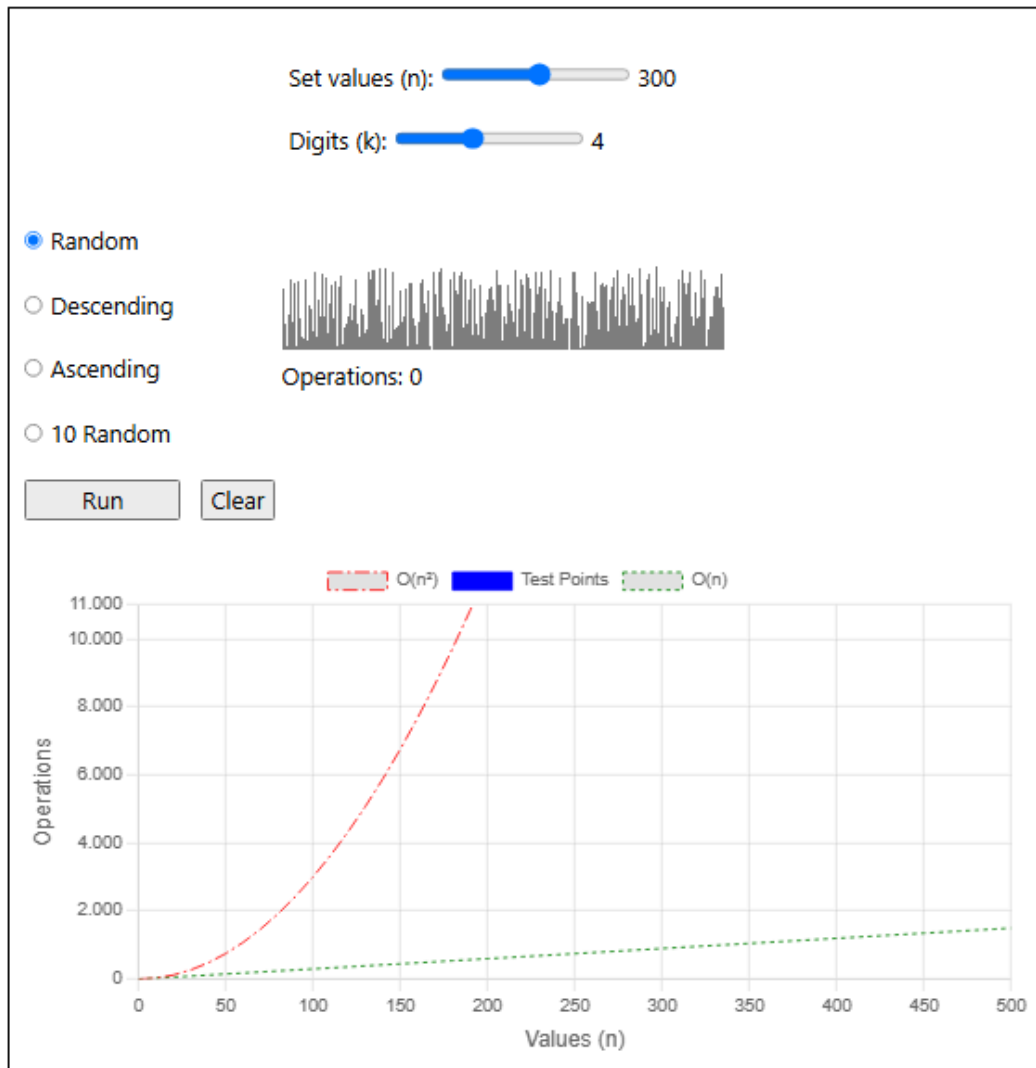
A complexidade temporal do algoritmo é dada por $\Theta(d \cdot (n+k))$, onde d representa a quantidade de dígitos do maior número da entrada, n é a quantidade de elementos a serem ordenados e k corresponde ao número de possíveis valores para cada dígito. Quando o valor de d permanece relativamente pequeno, o desempenho do algoritmo torna-se bastante eficiente, aproximando-se de um comportamento linear.

Uma das principais vantagens do Radix Sort em relação ao Counting Sort está relacionada ao tamanho do intervalo dos valores de entrada. O Counting Sort possui complexidade dependente diretamente do valor máximo possível (k), tornando-se inviável quando esse intervalo é muito grande. Já o Radix Sort divide o problema em várias etapas menores, processando apenas um dígito por vez. Dessa forma, mesmo quando os números possuem valores elevados, o algoritmo consegue manter um desempenho eficiente desde que a quantidade de dígitos não seja excessivamente grande.

Durante a minha pesquisa, foi encontrada uma discussão interessante no fórum Software Engineering Stack Exchange intitulada **"Why isn't radix sort used more often?"**. Essa indagação chamou atenção porque, teoricamente, o Radix Sort pode apresentar desempenho superior a algoritmos baseados em comparação. Entretanto, diversos fatores limitam sua utilização prática, como o consumo adicional de memória, a necessidade de conhecer previamente o formato dos dados, a maior complexidade de implementação e o fato de que algoritmos como Quick Sort e Merge Sort apresentam excelente desempenho em aplicações gerais. Assim, embora o Radix Sort seja extremamente eficiente em cenários específicos, ele não é a solução ideal para todos os problemas de ordenação.

Para que sua utilização seja prática, algumas condições devem ser observadas. O algoritmo é mais adequado para ordenar números inteiros, identificadores, códigos ou outros dados que possam ser divididos em dígitos ou caracteres de tamanho conhecido. Além disso, seu desempenho é mais vantajoso quando a quantidade de dígitos é limitada e quando o conjunto de dados possui grande volume de elementos.

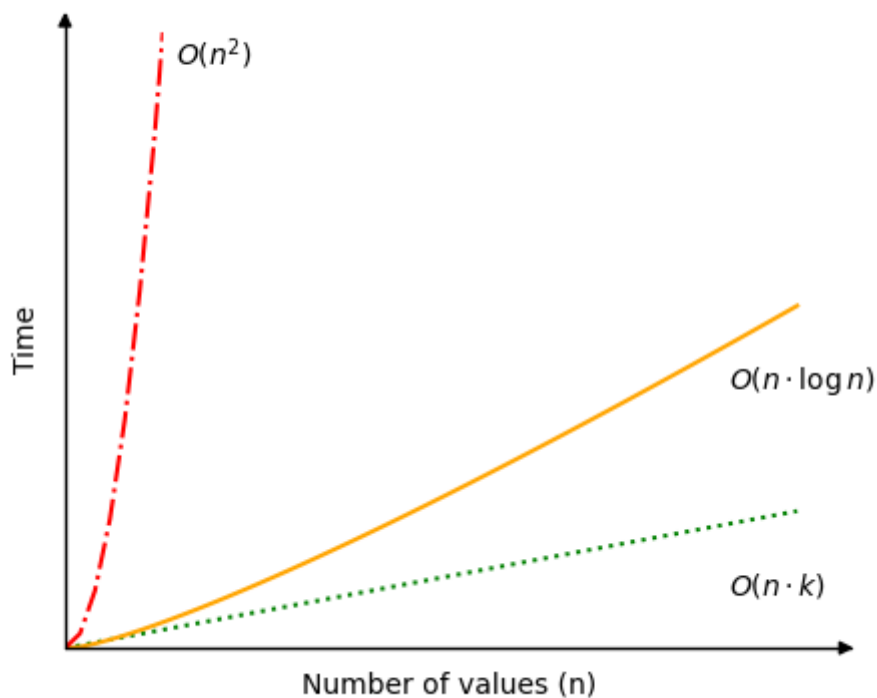
Para compreender melhor o funcionamento do algoritmo, foi utilizada a demonstração interativa disponibilizada pelo W3Schools, que permite acompanhar visualmente cada etapa da ordenação dos dígitos:



Fonte: https://www.w3schools.com/dsa/dsa_algo_radixsort.php

O melhor cenário ocorre quando existe uma grande quantidade de elementos para ordenar, mas os valores possuem poucos dígitos. Nessa situação, o valor de **k** pode ser considerado praticamente constante. Por exemplo, se houver mais de um milhão de números para ordenar e o maior deles for 999, serão necessárias apenas três passagens pelos dados. Assim, a complexidade **$O(n \cdot k)$** pode ser simplificada para **$O(n)$** , resultando em um desempenho muito eficiente.

Já o pior cenário ocorre quando a quantidade de dígitos do maior valor cresce proporcionalmente ao número de elementos. Embora essa situação seja pouco comum na prática, a complexidade pode chegar a **$O(n^2)$** . Nesse caso, o algoritmo precisa realizar um número muito maior de etapas de processamento, reduzindo sua eficiência.



Fonte: https://www.w3schools.com/dsa/dsa_algo_radixsort.php

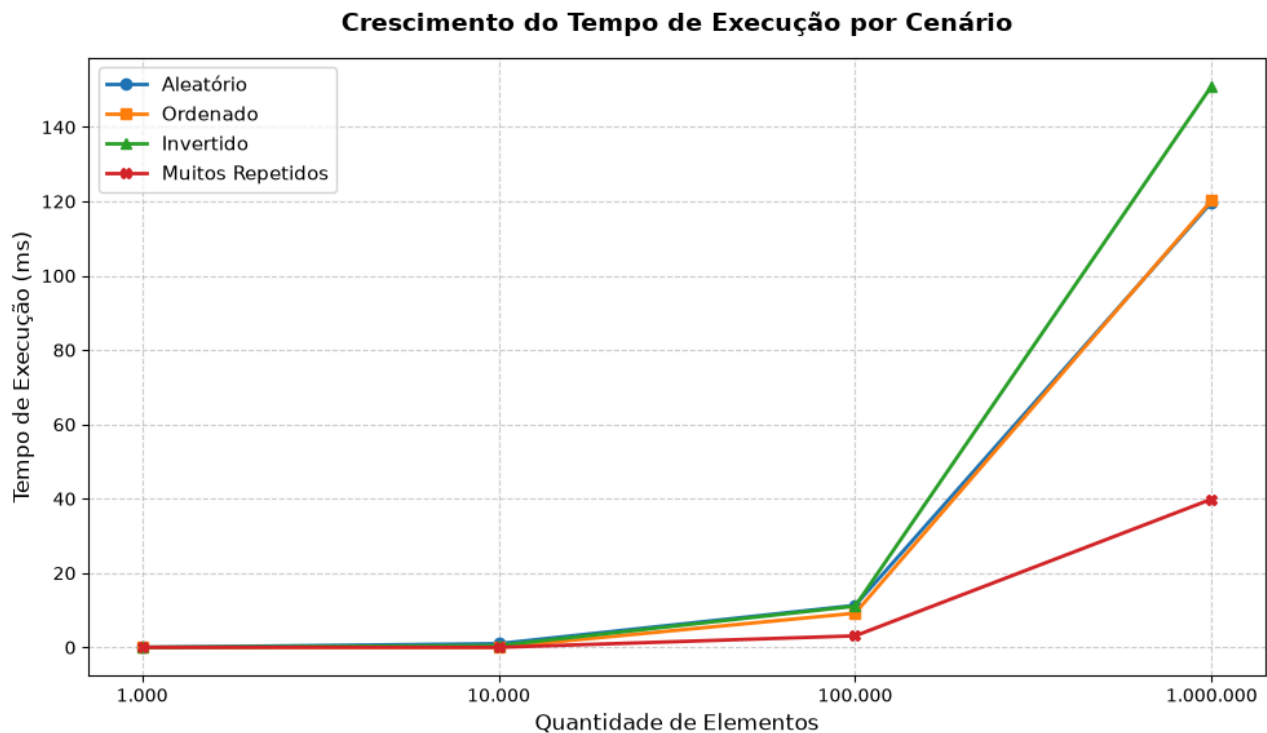
Entre esses dois extremos encontra-se o caso mais comum. Quando a quantidade de dígitos cresce aproximadamente de acordo com $\log(n)$, a complexidade torna-se $O(n \cdot \log n)$. Um exemplo seria ordenar um milhão de valores contendo cerca de seis dígitos cada. Nessa situação, o algoritmo mantém um desempenho bastante competitivo, o que explica sua utilização em aplicações que trabalham com grandes volumes de números inteiros.

Portanto, o desempenho do Radix Sort está diretamente relacionado à quantidade de dígitos dos valores processados. Quanto menor for o número de dígitos em relação à quantidade de elementos, mais próximo o algoritmo estará de um comportamento linear.

Após o estudo teórico, o algoritmo foi implementado em linguagem C# (código presente no github). Em seguida, foram realizados testes 10 utilizando diferentes distribuições de entrada, incluindo vetores aleatórios, vetores ordenados, vetores em ordem inversa e vetores com elementos repetidos. Foi-se calculada a média dos tempos obtidos, os resultados foram representados graficamente para facilitar a visualização do crescimento do tempo de execução em função da quantidade de elementos processados. Observou-se que o Radix Sort apresentou comportamento próximo ao esperado teoricamente, mantendo crescimento aproximadamente linear para valores com quantidade limitada de dígitos.

Quantidade de Elementos	Aleatório	Ordenado	Invertido	Muitos Repetidos
1.000	0,0 ms	0,0 ms	0,0 ms	0,0 ms
10.000	1,0 ms	0,0 ms	0,5 ms	0,0 ms
100.000	11,3 ms	9,2 ms	11,1 ms	3,1 ms
1.000.000	119,8 ms	120,3 ms	151,1 ms	39,9 ms

Fonte: Autoria própria



Fonte: Autoria própria

Bucket Sort:

O Bucket Sort é um algoritmo de ordenação que funciona de um jeito bem intuitivo, ele pega os elementos e os distribui em diferentes grupos, que chamamos de *buckets* (ou baldes), baseando-se nos seus valores. Depois que tudo está separado, cada balde é ordenado por si só e, no final, juntamos todos eles em sequência para entregar o vetor perfeitamente ordenado.

Na prática, podemos dividir esse processo em três grandes passos. Primeiro, os elementos são espalhados pelos baldes conforme a faixa de valor em que se encaixam. Em seguida, cada balde é ordenado individualmente, geralmente usando um algoritmo mais simples, como o *Insertion Sort*. Por fim, reunimos os dados de todos os baldes na ordem certa, gerando o resultado final.

O grande segredo do Bucket Sort é que o seu desempenho está totalmente amarrado à forma como os dados de entrada estão distribuídos. Se os elementos forem bem espalhados (uma distribuição uniforme), os baldes vão receber quantidades parecidas de itens, o que torna o processo incrivelmente rápido e eficiente.

Em termos de complexidade, o cenário médio dele é de $O(n + k)$, onde n é a quantidade de elementos e k é o número de baldes. Sob essas condições ideais, o algoritmo consegue entregar um desempenho muito próximo do linear, brilhando bastante quando precisamos lidar com grandes volumes de dados bem distribuídos.

Por outro lado, o pior cenário acontece se muitos elementos caírem em um único balde. Quando isso ocorre, a ordenação interna desse balde sobrecarrega o sistema e domina o tempo de execução, o que pode fazer a complexidade saltar para $O(n^2)$. É por isso que a eficiência dele depende tanto do comportamento dos dados.

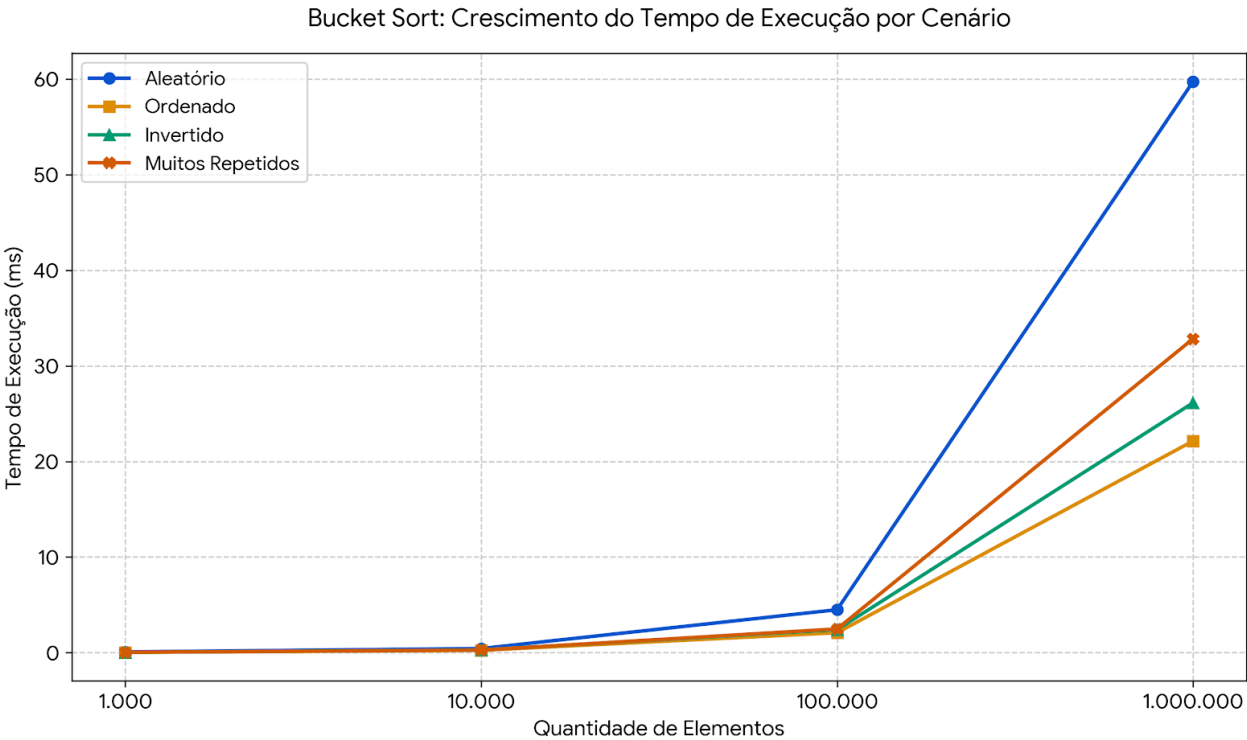
Durante o levantamento para este trabalho, encontrei uma discussão bem interessante no Stack Overflow chamada **"When should I choose Bucket Sort over other sorting algorithms?"** (Quando devo escolher o Bucket Sort em vez de outros algoritmos?). O debate reforça justamente que ele é a escolha ideal quando já conhecemos um pouco sobre a natureza dos dados. Se sabemos que a distribuição é uniforme, ele pode superar com folga os métodos tradicionais de ordenação. Mas, se os dados estiverem muito concentrados em uma única faixa de valores, a performance cai drasticamente.

Apesar disso, as fontes pesquisadas destacam duas grandes vantagens: a sua simplicidade conceitual e a versatilidade. Ele pode ser adaptado para diferentes tipos de dados, contanto que a gente consiga criar uma lógica eficiente para dividir os elementos entre os baldes. Para valer a pena na prática, o cenário ideal envolve dados distribuídos uniformemente em um intervalo já conhecido e uma boa estratégia de divisão.

Depois de consolidar essa base teórica, o algoritmo foi implementado em C# (com o código disponível no GitHub) e submetido a testes práticos. Foram avaliados cenários com vetores aleatórios, ordenados, invertidos e repletos de elementos repetidos. Para garantir a precisão, rodamos dez repetições de cada teste e calculamos a média dos tempos, gerando gráficos para visualizar como o algoritmo reage a cada situação.

Quantidade de Elementos	Aleatório	Ordenado	Invertido	Muitos Repetidos
1.000	0,080 ms	0,030 ms	0,031 ms	0,036 ms
10.000	0,426 ms	0,243 ms	0,280 ms	0,266 ms
100.000	4,492 ms	2,080 ms	2,399 ms	2,482 ms
1.000.000	59,728 ms	22,139 ms	26,133 ms	32,831 ms

Fonte: Autoria própria



Fonte: Autoria própria

No fim das contas, a prática confirmou exatamente o que a literatura previa, o sucesso do Bucket Sort está no equilíbrio. Quanto mais uniforme for a divisão dos dados entre os baldes, mais eficiente e veloz o algoritmo se mostra.

Conclusões:

Entre os **algoritmos baseados em comparação**, fica claro que o limite inferior de $O(n \log n)$ dita a eficiência máxima. Enquanto métodos elementares (como Bubble, Insertion e Selection) operam em tempo quadrático $O(n^2)$, algoritmos otimizados como o Merge Sort e o Heap Sort atingem a fronteira matemática ideal de $O(n \log n)$.

A tabela também destaca como os **algoritmos não comparativos** (Counting, Radix e Bucket Sort) conseguem quebrar essa barreira e operar em tempo linear ou quase linear. Eles alcançam isso explorando características específicas da entrada, como a divisão por dígitos ou a distribuição dos valores em intervalos.

Contudo, essa velocidade extra traz um custo: o **consumo de memória**. Enquanto métodos como o Heap Sort não exigem memória adicional $O(1)$, os algoritmos lineares precisam de espaço extra proporcional aos dados ($O(n)$ ou $O(n+k)$). Além disso, sua eficiência é altamente dependente da natureza dos dados, o Bucket Sort, por exemplo, pode despencar para $O(n^2)$ no pior caso se os elementos não tiverem uma distribuição uniforme.

Em conclusão, a escolha do algoritmo de ordenação ideal não se baseia apenas em buscar a menor complexidade matemática. Ela exige um equilíbrio cuidadoso entre o formato e a distribuição dos dados a serem processados, os limites de memória do sistema e a necessidade de preservar a ordem original de elementos iguais (estabilidade).

Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Memória	Estável
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Não
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Não
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Não
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	Sim
Radix Sort	$O(d \cdot (n+k))$	$O(d \cdot (n+k))$	$O(d \cdot (n+k))$	$O(n+k)$	Sim
Bucket Sort	$O(n)$	$O(n)$	$O(n^2)$	$O(n)$	Depende

Fonte: Autoria própria

Uma observação importante é que para ajuda em resolução de bugs em códigos de teste foi-se utilizado o auxílio de ferramentas de IA (mais especificamente o GEMINI IA).

Referências:

BAELDUNG. Radix Sort vs. Counting Sort vs. Bucket Sort. Disponível em: <https://www.baeldung.com/cs/radix-vs-counting-vs-bucket-sort>. Acesso em: 18 jun. 2026.

GEEKSFORGEES. Bucket Sort. Disponível em: <https://www.geeksforgeeks.org/dsa/bucket-sort-2/>. Acesso em: 18 jun. 2026.

IME USP. Basic sorting algorithms. Disponível em:
<https://www.ime.usp.br/~pf/algorithms/chapters/basic-sorting.html>. Acesso em: 20 jun. 2026.

MEILISEARCH. Bucket Sort Guide. Disponível em:
<https://www.meilisearch.com/blog/bucket-sort-guide>. Acesso em: 18 jun. 2026.

MIT OPENCOURSEWARE. Comparison Sort, Counting and Radix Sort. Disponível em:
https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/cb7df5fee9cb62e0810616491cdd1194_MIT6_006F11_rec07.pdf. Acesso em: 20 jun. 2026.

PROGRAMIZ. Bucket Sort Algorithm. Disponível em:
<https://www.programiz.com/dsa/bucket-sort>. Acesso em: 18 jun. 2026.

RESEARCHGATE. A Novel Time and Space Complexity Efficient Variant of Counting-Sort Algorithm. Disponível em:
https://www.researchgate.net/publication/338799866_A_Novel_Time_and_Space_Comple_xity_Efficient_Variant_of_Counting-Sort_Algorithm. Acesso em: 20 jun. 2026.

SOFTWARE ENGINEERING STACK EXCHANGE. Why isn't radix sort used more often? Disponível em:
<https://softwareengineering.stackexchange.com/questions/77529/why-isnt-radix-sort-used-more-often>. Acesso em: 18 jun. 2026.

STACK OVERFLOW. When should I choose Bucket Sort over other sorting algorithms? Disponível em:
https://stackoverflow.com/questions/31633391/when-should-i-choose-bucket-sort-over-oth_er-sorting-algorithms. Acesso em: 18 jun. 2026.

W3SCHOOLS. DSA Radix Sort. Disponível em:
https://www.w3schools.com/dsa/dsa_algo_radixsort.php. Acesso em: 18 jun. 2026.